

Whole-Protocol Admission for Restricted Python on RT Hardware

Anonymous Author(s)

Abstract

Ray-tracing APIs expose fast traversal hardware through a callback protocol whose correctness depends on more than callback types: build-input topology, payload layout, lifecycle order, continuation rules, and the identity of the executable code must agree. Existing Python interfaces make the protocol accessible, but do not establish that a restricted user program and all of its generated native components form one admitted execution.

We present RTDL, a bounded whole-protocol admission design for Python-authored ray-tracing computations. A typed callback schema, topology-specific trusted lowerers, executable-identity checks, and lifecycle gates jointly admit a small set of complete routes rather than accepting callbacks in isolation. The current artifact supports two stable public constructors and a bounded route corpus covering triangle, custom-AABB, round-linear-curve, and sphere leaves; this is not arbitrary Python or topology-generic lowering. A prospective sealed-snapshot exam selected a sphere any-hit counting composition and produced two real OptiX launches with 12/12 oracle checks. A separate checker, which does not import the compiler, validates five generated-code property classes in 20 registered instances, but remains a finite target-structure checker.

Across two NVIDIA GPU generations, we evaluate 160 registered cells containing 20,480 timed samples. At the prepared endpoint, public ahead-of-time RTDL median latency is $1.077\text{--}1.175\times$ a direct CUDA/OptiX implementation. First-result measurements are adverse: post-import RTDL is $1.560\text{--}1.837\times$ a strong PyOptiX device-continuation baseline by median, with a $2.377\times$ worst block. These results delimit the contribution: bounded protocol admission with near-direct prepared latency on two tasks, not intrinsic speedup, general lowering, or demonstrated usability.

CCS Concepts: • Software and its engineering → Compilers.

Keywords: ray tracing, restricted Python, protocol admission, GPU compilation

ACM Reference Format:

Anonymous Author(s). 2027. Whole-Protocol Admission for Restricted Python on RT Hardware. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO '27)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern ray-tracing systems expose programmable stages around fixed-function traversal. OptiX, Vulkan ray tracing, and DirectX Raytracing let programs choose geometry representations, payload conventions, hit behavior, and launch topology [9–11, 14]. This flexibility has enabled non-rendering workloads to use RT cores, but it also creates an unusual language boundary. A callback can be locally well typed and still be unsafe or semantically wrong because it is paired with the wrong build-input kind, stale generated code, an incompatible payload, or an invalid lifecycle transition.

Python bindings reduce API friction, but they do not eliminate this protocol problem. A Python user may assemble modules, program groups, shader-binding records, acceleration structures, streams, and buffers whose compatibility is represented only by convention. Conversely, a compiler that validates a restricted callback body but delegates the surrounding protocol to hand-written wrappers has proved only local compilation. The research question in this work is therefore:

Can a restricted Python system admit an entire bounded ray-tracing protocol, including generated native code, executable identity, and lifecycle state, while retaining prepared-execution latency close to direct CUDA and OptiX?

We answer for a deliberately narrow domain. RTDL accepts a typed, restricted callback representation and maps admitted programs to compiler-owned, topology-specific lowerers. It records the source, PTX, ABI, and native symbols that constitute an executable route; checks lifecycle transitions before launch; exposes status before output; and supplies an independent offline checker for selected generated-code properties. The canonical plan is a non-executable specification. Executable lowering remains in the trusted topology-specific implementation. This distinction is central: our evidence does not establish an arbitrary callback compiler.

This paper makes four contributions:

1. A *whole-protocol admission model* that binds callback effects, topology, ABI, executable identity, and lifecycle state into one bounded route decision.
2. A concrete Python-to-OptiX implementation with two stable public constructors, four leaf kinds in the bounded route corpus, and explicit topology-specific lowerers in the trusted computing base (TCB).

Table 1. Whole-protocol obligations in the current system.

Obligation	Admission question
Role/effect	Is each operation legal in this callback role?
Semantic ABI	Do payload and result meanings agree end to end?
Physical ABI	Do widths, alignments, slots, and row layouts agree?
Topology	Does the callback set match the build-input and leaf kind?
Identity/state	Is the checked executable the loaded executable, and is every state transition legal?

3. A finite, compiler-independent checker and mutation evidence that expose what is checked, what remains trusted, and where the current guarantees stop.
4. A preregistered two-generation evaluation of two tasks: prepared public latency is $1.077\text{--}1.175\times$ Direct, while adverse first-result and incomplete-receipt results constrain the claim.

2 The Protocol Mismatch

2.1 Why callback typing is insufficient

A ray callback runs inside a protocol distributed across host and device code. Its legal effects depend on its role: an any-hit callback may ignore or terminate an intersection, whereas a miss callback cannot. Its payload fields must agree with every producer and consumer. Its program group must match the acceleration-structure leaf kind. The launch must occur only after module, pipeline, shader-binding-table, geometry, and output state have been prepared. Finally, the code checked by the compiler must be the code loaded by the runtime.

These are related to typestate, interface automata, and session-style protocol reasoning [2, 7, 17], but the FFI and generated-code boundary adds an identity problem familiar from linking and foreign interfaces [4, 15]. Table 1 summarizes the obligations used here.

2.2 A concrete semantic-ABI failure

Consider two acceleration-structure primitives at physical positions 0 and 1 whose application identifiers are 10 and 20. Queries 100 and 101 should emit the pairs (100, 10) and (101, 20). In an executed defective route, the intersection callback instead placed the primitive position in attribute slot zero. The any-hit callback consumed that slot as an application identifier and returned (100, 0) and (101, 1). Every value remained a valid u32; the acceleration structure, module, program groups, pipeline, shader binding table, and launch were legal. Typed OptiX and construction routes reached launch and returned the wrong rows.

The defect is not a machine-type error but a disagreement over what the bits mean. In RTDL, a fixed route contract

says that attribute slot zero denotes an application identifier, while the compiled target description records the actual row source. Admission compares these authorities and rejects the disagreement before launch. The system does not infer which objects should match, and an application oracle is still necessary. Admission instead rejects the broken producer–consumer agreement independently of whether a test input happens to make primitive positions and identifiers differ.

2.3 Failure model

We distinguish four outcomes. *Reject* means admission fails before a launch. *Status failure* means execution reports failure and output is not published. *Wrong output* means a worker reports success but an independent oracle disagrees. *Unverified output* means a value exists without the required receipt or oracle evidence. The distinction prevents a status gate from being described as numerical correctness and prevents a generated plan from being described as an executable lowering.

The implementation has a double-fault caveat: teardown failure while a provider releases retry ownership can replace a primary exception. No public output is then released, and we did not observe this event, but the diagnostic provenance can be degraded. Similarly, the fork guard is enforced for Python-managed use; bypassing the public layer and invoking native state after a fork is outside the admitted contract.

3 Bounded Language and Admission

3.1 Callback representation

The source language is restricted Python, not arbitrary Python. Source is parsed through an AST allowlist and is not imported or executed for discovery. The front end resolves declared immutable records, frozen constants, same-unit acyclic helpers, and selected intrinsics, then produces canonical typed IR. Deserialization reconstructs typed objects and reruns verification; serialized IR is not itself an authority. Table 2 summarizes the representation.

The current resource contract admits at most 32 payload slots and eight attribute slots, trace depth one, callable depth zero, bounded helper depth, and statically bounded iteration. The numeric policy requires explicit binary32 behavior and fail-closed handling of prohibited integer overflow and nonfinite effects. These are language and resource checks; they do not prove that a chosen ray encoding computes the intended application function.

3.2 Role-indexed effects

Seven language roles participate in an admitted unit. bounds runs during preparation; `make_ray` and `finalize` are language leaves invoked by a trusted ray-generation wrapper; and `intersection`, `any_hit`, `closest_hit`, and `miss` map to target-stage behavior. User code cannot invoke raw traversal. Instead, each role returns one typed effect from a

Table 2. Implemented bounded representation. Rejected forms have no fallback lowering.

Form	Accepted subset	Rejected boundary
Types	Booleans; fixed-width integers and floats; 2–4-lane vectors; tuples; acyclic immutable records; checked read-only views	Raw pointers, mutable containers, opaque objects, and recursive records
Expressions	Typed names and literals, record fields, checked indexing, arithmetic, comparisons, bit operations, constructors, selected pure intrinsics, and bounded helper calls	Reflection, dynamic dispatch, allocation, foreign calls, and unknown attributes
Statements	Typed binding, loop-local type-preserving update, two-sided conditionals, statically bounded loops, role-effect return, and helper-value return	Dynamic loops, recursion, exceptions, generators, async control, and not-all-paths-return
Manifest	Payload/output records, machine attributes, constants, numeric policy, resource budget, topology, continuation, and linkage	Self-minted physical authority, unreviewed production linkage, and unsupported budgets
Program	Canonical source and IR identities, records, functions, manifest, and separately derived effect digest	Unknown fields, malformed schema, stale identities, or unverified deserialization

role-specific set: bounds returns an AABB, ray construction returns a trace request, intersection returns hit or no-hit, hit callbacks return payload plus continue/ignore/terminate control, miss returns payload, and finalize returns an output.

Topology makes these requirements relational. Custom AABBs require bounds, ray construction, intersection, miss, finalize, and at least one hit role. Built-in triangles forbid bounds and intersection because hardware owns them, but retain ray construction, miss, finalize, and at least one hit role. An any-hit role also requires an explicit delivery and continuation contract; the presence of that declaration does not prove order independence for arbitrary callback logic.

3.3 Fail-closed judgments

Expression checking derives $\Gamma \vdash e : \tau$. Statement checking also returns the output environment, all-paths-return bit, conservative static work, and observed effects:

$$\Gamma; r \vdash s \Rightarrow \langle \Gamma', q, n, \epsilon \rangle. \quad (1)$$

For each role r , verification enforces its exact signature and

$$M \vdash F_r : \epsilon \quad \text{with} \quad \epsilon \subseteq \text{AllowedEffects}(r). \quad (2)$$

Names are single-assignment except for type-preserving updates inside static loops; loop locals cannot escape; both non-returning branches define the same typed names; and every role/helper path returns. Whole-unit checking validates schema/source identity, acyclic records and helpers, numeric and resource budgets, role cardinality, topology, linkage, and required external physical authority:

$$\mathcal{G}; \mathcal{R} \vdash Q \Downarrow \text{Verified}(h_{\text{IR}}, h_{\text{effects}}). \quad (3)$$

Here $\mathcal{G}$ is an out-of-program geometry authority and $\mathcal{R}$ is the compiler-owned role topology. The judgment is explanatory notation for implemented checks, not a mechanized soundness theorem.

Admission is conjunctive. A route is accepted only if its schema is valid, its roles permit all effects, its semantic and physical ABIs agree, a trusted lowerer exists for its topology, the generated executable identity is bound, and the runtime lifecycle can reach the prepared state. Failure of any one predicate rejects the route; there is no “best effort” projection into a nearby executable.

3.4 Layered whole-protocol admission

Table 3 identifies where each obligation originates. This separation matters because target evidence is regenerated within one trusted implementation; it is not an independent proof of compiler correctness.

3.5 Plan versus executable lowering

The canonical planner normalizes the admitted schema and emits a declarative plan. That plan is intentionally marked non-executable. It does not synthesize arbitrary CUDA or OptiX callbacks. Instead, an accepted route names a compiler-owned, topology-specific lowerer and wrapper. The lowerer emits source and PTX, binds native symbols, constructs the program groups and shader-binding table, and records their identities. This design supplies a common admission surface while keeping concrete backend knowledge visible in the TCB.

The distinction addresses a common overclaim. Sharing schema validation and lifecycle machinery does not imply that the device implementation is generic. In the prospective composition described in Section 5.2, the new sphere route required approximately 2,635 lines of topology-specific code and 28 modified compiler lines. That is evidence that the framework can admit a new trusted route without changing three sealed shared files, not evidence that a new topology is lowered automatically.

Table 3. Authorities compared by whole-protocol admission.

Obligation	Declaration authority	Target/runtime evidence	Exact limit
Role/effect	Verified role topology and returned effects	Reverified compiled callback ABI variants	Common compiler TCB; no arbitrary effect inference
Semantic ABI	Fixed-route nominal slot and row-source contract plus machine types	Generated route row sources and callback ABI	Evaluated attribute slot zero; generic u32 has no nominal meaning
Physical ABI	Typed plan for geometry, buffers, hit channel, result, and reducer	Rebuilt target facts and bound storage	Exact-plan admission, not arbitrary physical synthesis
Continuation	Completeness/capacity policy and required status-before-use order	Status vocabulary, compiled continuation, runtime status gate	Bounded supported modes; not a confluence theorem
Identity/lifecycle	Materializer-selected source/PTX/provider identity and state machine	Loaded symbols, prepared objects, execution status, and optional receipt	Identity binding is not semantic correctness

3.6 Executable identity and lifecycle

The system binds hashes and shape metadata for generated source, PTX, ABI, native provider symbols, and the public route. Runtime preparation checks the binding before execution. The admitted lifecycle is:

static verify → materialize/bind → prepare → execute/status gate → output → receipt.

The final public path checks native and compact status synchronously and the experiment checks every returned output against an oracle before declaring a worker successful. It does *not* rehash files from disk on every launch. Moreover, the original design goal of a detailed receipt for every timed execution is not met: 4,096 timed executions in the main RTDL arm are accompanied by 32 separate post-loop detailed diagnostic receipts. We therefore claim synchronous status and output gating, not per-execution detailed receipt coverage.

4 Implementation Scope

The implementation uses Python for the public authoring surface and C++/CUDA with OptiX for trusted lowering and execution. This is a compiler/runtime prototype rather than a pure-Python implementation. Table 4 separates stable public constructors from the broader bounded route corpus.

The route corpus is broader than the stable facade but narrower than OptiX. Unsupported geometry options, motion modes, callback combinations, and payload shapes remain outside the contract. Extension requires adding or auditing a trusted lowerer; a matching schema alone is insufficient.

5 Evaluation

We ask four questions: (1) do whole-protocol mutations reach the intended gates; (2) can a sealed shared framework admit one selected composition; (3) does a separate checker validate concrete generated artifacts without trusting the compiler’s own projection; and (4) what performance cost does the public route impose at registered endpoints?

Table 4. Implemented scope; presence is not complete feature coverage.

Surface	Current evidence
Stable facade	Two constructors: custom-AABB bounded canonical relation; built-in triangle checked u64 reduction
Stable kind presence	2 of 6 OptiX build-input enum values; 2 of 4 leaf kinds
Bounded V4 corpus	4 of 6 build-input enum values; all 4 leaf kinds (triangle, custom, curve, sphere)
User-authored template	One bounded built-in-triangle template
Prospective composition	One sealed-snapshot sphere composition
Human usability	No independent-user study or timing

5.1 Earlier liveness and residual evidence

For two accepted fixed contracts, 19 single-leaf mutations exercised 19/19 expected gates: seven role-and-effect, one semantic-ABI, seven physical-ABI, two continuation, and two identity mutations. This denominator is limited to those two contracts and those mutations; it is not combined with the later composition or checker experiments.

We also examined typed residual failures against OptiX and OWL-style execution paths [12, 13]. Three residuals produced complete wrong-output witnesses. A fourth supplied only a partial status-gated continuation/completeness witness. Checked executable identity adds supporting evidence. These are concrete failure observations, not five proved guarantees or general soundness theorems.

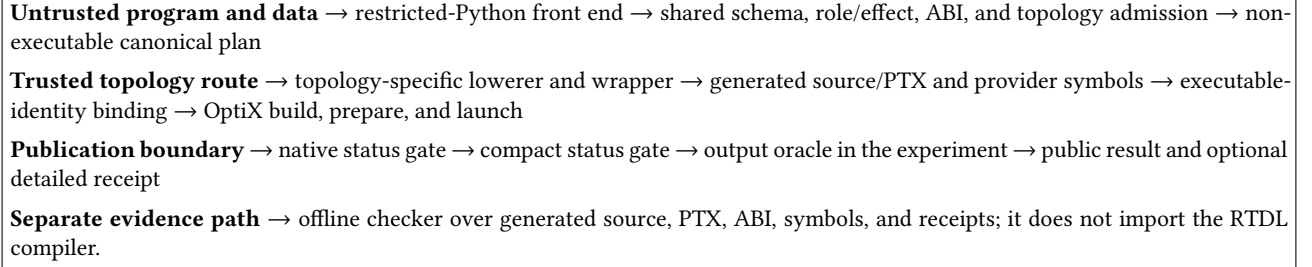


Figure 1. Architecture and trust boundary. Shared admission does not replace the topology-specific lowerer; that lowerer remains in the TCB.

5.2 Sealed-snapshot composition exam

Before revealing a future public randomness pulse, we fixed an author-defined challenge table of ten rows: seven built-in four-role rows and three custom six-role rows. All rows used already-supported primitive families; the three continuation variants all used a per-query count relation. The round-linear-curve any-hit-terminate row remained an eligible candidate; it was not selected. The pulse selected `builtin_sphere::any_hit_count_continue_u64_per_query`.

The artifact’s private custody record identifies the exact sealed historical source. Implementation changed no bytes in the three frozen schema, admission, and lifecycle files. The resulting route performed two true OptiX launches and passed 12/12 rational-oracle rows. The result demonstrates framework-level reuse of admission, identity, and lifecycle machinery for this selected composition. It does not establish an unbiased new application: the authors defined the challenge domain, every row recombined known primitives and topologies, and the selected route required substantial new topology-specific implementation.

5.3 Independent finite checker

The offline checker imports no RTDL module and rejects compiler-supplied projections as authority. It independently reads generated source, PTX/ABI metadata, symbols, and receipts. Across three route groups and four modes, it applied five property classes 20 times using 15 unique mutations. It also partially evaluates generated target structure rather than trusting labels in the compiler output.

This checker is finite, not a verifier for arbitrary device control flow. During review, an out-of-authority in-memory probe inserted an unconditional early return and still passed selected effect/status helper checks. The result therefore supports specialized target-structure validation only. General control-flow, numerical, and refinement soundness remain open; tools such as GPUVerify illustrate the substantially stronger proof obligation [1].

5.4 Performance methodology

We measured two exact tasks. *Relation* uses 4,096 objects and 4,096 queries, returns 4,096 canonical u32 pair rows, and performs two true OptiX traversals. *Triangle* uses 16,384 primitives and 16,384 queries, performs one true traversal, and returns the checked scalar 65,530.

Five arms isolate different boundaries:

- A current RTDL ahead-of-time public path at measured source M;
- B idiomatic pinned PyOptiX-compatible baseline;
- C strong PyOptiX-compatible baseline with device continuation;
- D direct CUDA/OptiX implementation;
- E frozen RTDL predecessor immediately before M.

We ran the same registered matrix on an RTX 4090 (Ada) and RTX 3090 (Ampere). Each generation has eight blocks, two tasks, and five arms: 80 cells, with 128 steady samples per cell. Across both generations this yields 160 cells and 20,480 samples. A reported ratio is the median of eight within-block ratios of integer nanosecond measurements; lower is better. Raw times are not compared across machines.

The registered primary criterion was prepared A/D median ≤ 1.20 . There was no registered A/D maximum-block gate. A registered A/C implementation-entry criterion used median ≤ 1.20 and maximum ≤ 1.35 , but the endpoint was revised after an adverse result and both available first-result endpoints remain import/lifecycle-confounded. We therefore make no positive entry-latency claim. The registered A/E prepared criterion was ≤ 1.05 ; C/B competence used ≤ 1.05 .

5.5 Prepared execution results

Table 5 reports the main observation. The public AOT path is 1.077–1.175× the direct implementation by median. The triangle maximum block on Ada is 1.211×; because no maximum A/D gate was registered, it is reported as dispersion rather than converted into a pass/fail criterion.

Ahead-of-time cache-hit latency was measured separately over 20 observations. Median hits were 58.3–78.2 ms and hit/cold ratios were approximately 1.04–2.02%. A cache

Table 5. Prepared public RTDL versus direct CUDA/OptiX (A/D). Ratio is lower-is-better; times are medians of worker medians.

GPU	Task	A (ms)	D (ms)	Median / max
RTX 4090	Relation	0.2810	0.2612	1.0769 / 1.0923
RTX 4090	Triangle	0.0598	0.0510	1.1751 / 1.2110
RTX 3090	Relation	0.2402	0.2198	1.0948 / 1.1188
RTX 3090	Triangle	0.0608	0.0536	1.1336 / 1.1427

hit still includes cache lookup, validation, and load; it is not an initial compilation and is not part of the prepared kernel endpoint.

5.6 Adverse lifecycle results

Table 6 preserves the unfavorable results. Every A/C post-import median is adverse, from 1.560× to 1.837×, and the worst formal block is 2.377×. The entry endpoint itself was revised after the adverse result, so neither entry nor post-import values support a confirmatory claim. Against predecessor E, A also regresses at first result by approximately 8–22% at entry and 16–31% post-import, despite the registered prepared A/E comparison being favorable. This is evidence that protocol machinery and Python/native lifecycle work remain material before steady state.

Arm C outperformed B in all four formal steady comparisons (C/B ratios 0.221–0.654), establishing that B alone would be a weak baseline for this experiment. It does not establish C as globally optimal. We also recorded 1,024 timer-instrumentation endpoints for A only. Those observations quantify A’s own timer overhead but do not qualify or symmetrically instrument B, C, D, or E.

No wrong output was observed in the final GPU samples. This statement is not a correctness proof and does not repair the detailed-receipt shortfall described in Section 3.6.

6 Programming Responsibility and Generality

The design separates three responsibilities. Users provide bounded callback intent and application data. Shared RTDL machinery validates schemas, cross-role effects, ABI compatibility, executable identity, and lifecycle. Trusted per-topology lowerers remain responsible for concrete CUDA and OptiX code generation and binding. The runtime refuses to publish output when its status gates fail, while experiments may add application-level oracles.

This decomposition is reusable beyond the present implementation. A system crossing from a managed language to an accelerator can adopt whole-protocol admission: name roles and effects, make semantic and physical ABI explicit, bind checked artifacts to loaded artifacts, and gate state transitions. The same principles can inform other Python

GPU systems, shader languages, and foreign-function interfaces. What does not automatically transfer is the topology-specific lowerer or the finite checker vocabulary. Those components must be re-established for each backend.

7 Related Work

OptiX established a programmable ray-tracing pipeline [14], and current OptiX, Vulkan, and DXR interfaces expose explicit pipeline and shader-group contracts [9–11]. PyOptiX-style bindings and OWL improve host-language access and construction ergonomics [12, 13]; our focus is not replacing those APIs but admitting a bounded cross-stage protocol and binding it to executable identity.

Shader and heterogeneous programming systems provide richer compilation abstractions. Slang’s capability system and modular shader work address composition and target restrictions [5, 6, 16]. Dr.Jit, LuisaRender, and CrossRT explore staged or cross-language generation for graphics and simulation [3, 8, 18]. RTDL differs in its narrow emphasis on admitting the surrounding RT protocol, but its current topology-specific lowerers are less general than those systems’ broader compilation ambitions.

The protocol framing draws on typestate and interface theories [2, 7, 17] and on typed linking and FFI work [4, 15]. Our finite artifact checker is empirical and structural; it should not be confused with formal GPU verification such as GPUVerify [1].

8 Artifact

The candidate artifact is an evidence-only, nine-member archive generated from the final frozen tooling snapshot. It contains an anonymous evidence projection, summary, manifest, documentation, dependency/component inventory, and a Python verifier using only the standard library. Private provenance and export receipts remain in the separate custodial record and are not distributed in the anonymous archive. The projection records 160 formal cells, 20,480 steady samples, 1,024 A-only instrumentation endpoints, 20 AOT samples, and eight competence records.

Two clean exports were byte-identical. Replays in two fresh directories, one with spaces, passed under isolated normal and optimized Python modes. The archive bundles no third-party source, binary, dataset, CUDA/OptiX header, driver, key, or raw GPU archive. It therefore supports offline verification of the submitted evidence, not GPU rerun, product installation, or full reconstruction of private historical authorities. CUDA, OptiX, drivers, and GPU hardware are external requirements for new execution.

Table 6. Lifecycle ratios. A/C compares current RTDL with the strong device-continuation baseline; A/E compares current with the frozen predecessor. Ratios are lower-is-better.

GPU	Task	A/C entry	A/C post median	A/C post range	A/E entry	A/E post	A/E prepared
RTX 4090	Relation	0.654	1.749	1.725–1.866	1.192	1.305	0.584
RTX 4090	Triangle	0.642	1.560	1.527–1.639	1.080	1.169	0.903
RTX 3090	Relation	0.681	1.837	1.816–2.377	1.217	1.262	0.608
RTX 3090	Triangle	0.618	1.637	1.608–1.653	1.138	1.163	0.922

9 Threats to Validity

Construct validity. Prepared A/D measures the public steady path after setup; it does not measure interactive authoring, cold compilation, or first use. Direct D is a useful lower-bound implementation, not an application framework. Strong C is contract-equivalent at the output boundary but is neither byte-identical nor operation-identical to A. A-only instrumentation does not qualify other arms.

Internal validity. Both tasks were used during system adaptation and tuning; they are not unseen confirmatory workloads. The implementation-entry endpoint was revised after an adverse observation, and both first-result endpoints include import and lifecycle effects. We preserve those outcomes rather than reinterpret them as a pass. The provider double-fault and Python-only fork boundary remain documented risks.

External validity. Measurements cover two consumer NVIDIA generations and two exact tasks. We do not compare raw latency across machines. The prospective challenge domain was author defined, used known primitive and topology ingredients, and selected one composition. There is no independent human authoring study, no prevalence measurement, and no evidence for arbitrary Python or arbitrary Callback IR.

Implementation trust. Approximately 2,635 lines of new topology-specific code were required for the selected sphere route. These lowerers and wrappers are in the TCB. The independent checker covers finite target structures and missed an out-of-authority early-return probe for selected helper checks. Detailed per-execution receipts were not produced for the 4,096 timed A executions.

Artifact scope. The anonymous projection is derived evidence, not the raw private archives. Its verifier checks internal identities, counts, registered formulas, and disclosed rows, but cannot independently recreate GPU runs or historical custody. Private provenance remains available for confidential audit.

10 Conclusion

RTDL demonstrates bounded admission for a complete ray-tracing protocol, instead of checking callbacks in isolation.

Shared schema, identity, and lifecycle machinery supports two stable public constructors and a four-leaf bounded corpus, while executable lowering remains topology-specific and trusted. Mutation, sealed-snapshot composition, and finite-checker evidence exercise this boundary without establishing general compiler soundness.

For two exact tasks on Ada and Ampere GPUs, the prepared public route costs $1.077\text{--}1.175\times$ direct CUDA/OptiX latency by median. First-result results are materially worse, with post-import A/C medians up to $1.837\times$ and a $2.377\times$ worst block. The contribution is bounded whole-protocol admission with near-direct prepared latency and explicit TCB, receipt, generality, and lifecycle limits. Removing those limits is the next research problem.

References

- [1] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, New York, NY, USA, 113–132. <https://doi.org/10.1145/2384616.2384625>
- [2] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 109–120. <https://doi.org/10.1145/503209.503226>
- [3] Vladimir Frolov, Vadim Sanzharov, Garifullin Albert, Maxim Raenchuk, and Alexei Voloboy. 2024. CrossRT: A Cross Platform Programming Technology for Hardware-Accelerated Ray Tracing in CG and CV Applications. arXiv:2409.12617. <https://doi.org/10.48550/arXiv.2409.12617>
- [4] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 62–72. <https://doi.org/10.1145/1065010.1065019>
- [5] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Transactions on Graphics* 37, 4, Article 141 (2018), 13 pages. <https://doi.org/10.1145/3197517.3201380>
- [6] Yong He, Theresa Foley, Teguh Hofstee, Haomin Long, and Kayvon Fatahalian. 2017. Shader Components: Modular and High Performance Shader Development. *ACM Transactions on Graphics* 36, 4, Article 100 (2017), 11 pages. <https://doi.org/10.1145/3072959.3073648>
- [7] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1, Article 9 (2016), 67 pages. <https://doi.org/10.1145/2827695>
- [8] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. 2022. DrJit: A Just-In-Time Compiler for Differentiable Rendering. *ACM Transactions on Graphics* 41, 4 (2022), 1–19. <https://doi.org/10.1145/3544444>

- 1145/3528223.3530099
- [9] Khronos Group. 2026. Vulkan Specification: Shader Interfaces. <https://docs.vulkan.org/spec/latest/chapters/interfaces.html>.
- [10] Microsoft. 2026. DirectX Raytracing Functional Specification: Payload Access Qualifiers. <https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html>.
- [11] NVIDIA. 2025. *NVIDIA OptiX 9.0 Programming Guide*. NVIDIA. https://raytracing-docs.nvidia.com/optix9/guide/optix_guide.250130.A4.pdf Payload types and per-stage payload semantics.
- [12] NVIDIA. 2026. PyOptiX: Complete Python Bindings for the OptiX Host API. <https://github.com/NVIDIA/otk-pyoptix>. Pinned experiment source commit 3144f224c0fd18733925faf3d8fb82c7376b8dcf.
- [13] NVIDIA, Ingo Wald, Stefan Zellmann, and contributors. 2026. OWL: The OptiX Wrappers Library. <https://github.com/NVIDIA/OWL>. Experiment pins upstream commit df7390b16bce5244b7352ca6d3e320f838297072.
- [14] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4, Article 66 (2010), 13 pages. <https://doi.org/10.1145/1778765.1778803>
- [15] Daniel Patterson and Amal Ahmed. 2017. Linking Types for Multi-Language Software: Have Your Cake and Eat It Too. In *2nd Summit on Advances in Programming Languages (Leibniz International Proceedings in Informatics, Vol. 71)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 12:1–12:15. <https://doi.org/10.4230/LIPIcs.SNAPL.2017.12>
- [16] Slang Project. 2026. Capabilities—Slang User’s Guide. <https://shader-slang.org/slang/user-guide/capabilities>.
- [17] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. <https://doi.org/10.1109/TSE.1986.6312929>
- [18] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Transactions on Graphics* 41, 6, Article 232 (2022), 19 pages. <https://doi.org/10.1145/3550454.3555463>